Numerische Mathematik 11, 181—195 (1968)

Handbook Series Linear Algebra

Householder's Tridiagonalization of a Symmetric Matrix*

Contributed by
R. S. MARTIN, C. REINSCH, and J. H. WILKINSON

1. Theoretical Background

In an early paper in this series [4] HOUSEHOLDER'S algorithm for the tridiagonalization of a real symmetric matrix was discussed. In the light of experience gained since its publication and in view of its importance it seems worthwhile to issue improved versions of the procedure given there. More than one variant is given since the most efficient form of the procedure depends on the method used to solve the eigenproblem of the derived tridiagonal matrix.

The symmetric matrix $A=A_1$ is reduced to the symmetric tridiagonal matrix A_{n-1} by $n-2$ orthogonal transformations. The notation is essentially the same as in the earlier paper [4] and the method has been very fully expounded in [6]. We therefore content ourselves with a statement of the following relations defining the algorithm.

$$A_{i+1}=P_iA_iP_i, \quad i=1, 2, \ldots, n-2, \tag{1}$$

where

$$P_i=I-u_iu_i^T/H_i, \quad H_i=\tfrac{1}{2}u_i^Tu_i, \tag{2}$$

$$u_i^T=[a_{i,1}^{(i)}; a_{i,2}^{(i)}; \ldots; a_{i,l-2}^{(i)}; a_{i,l-1}^{(i)}\pm\sigma_i^{\frac{1}{2}}; 0; \ldots; 0], \tag{3}$$

where $l=n-i+1$ and

$$\sigma_i=(a_{i,1}^{(i)})^2+(a_{i,2}^{(i)})^2+\cdots+(a_{i,l-1}^{(i)})^2, \tag{4}$$

$$p_i=A_iu_i/H_i, \quad K_i=u_i^Tp_i/2H_i, \tag{5}$$

$$q_i=p_i-K_iu_i, \tag{6}$$

$$\begin{aligned}A_{i+1}&=(I-u_iu_i^T/H_i)A_i(I-u_iu_i^T/H_i)\\&=A_i-u_iq_i^T-q_iu_i^T.\end{aligned} \tag{7}$$

The matrix A_i is of tridiagonal form in its last $i-1$ rows and columns. From the form of A_i and u_i it is evident that p_i is of the form

$$p_i^T=[p_{i,1}; p_{i,2}; \ldots; p_{i,l-1}; p_{i,l}; 0; \ldots; 0] \tag{8}$$

* *Editor's note.* In this fascicle, prepublication of algorithms from the Linear Algebra series of the Handbook for Automatic Computation is continued. Algorithms are published in ALGOL 60 reference language as approved by the IFIP. Contributions in this series should be styled after the most recently published ones.

so that q_i^T is given by

$$q_i^T = [p_{i,1} - K_i u_{i,1};\ p_{i,2} - K_i u_{i,2};\ \ldots;\ p_{i,l-1} - K_i u_{i,l-1};\ p_{i,l};\ 0;\ \ldots;\ 0]. \quad (9)$$

If z is an eigenvector of the tridiagonal matrix A_{n-1} then $P_1 P_2 \ldots P_{n-2} z$ is an eigenvector of A_1.

2. Applicability

The procedures *tred 1, tred 2, tred 3* all give the Householder reduction of a real symmetric matrix A_1 to a symmetric tridiagonal matrix A_{n-1}. They can be used for arbitrary real symmetric matrices, possibly with multiple or pathologically close eigenvalues. The selection of the appropriate procedure depends on the following considerations.

tred 1 stores A_1 in a square $n \times n$ array and can preserve full information on A_1 if required for use in computing residuals and or Rayleigh quotients. It should be used if eigenvalues only are required or if certain selected eigenvalues and the corresponding eigenvectors are to be determined via the procedures *bisect* [1] and *tridiinverse iteration* [5], (but improved version to be published [7]). *tred 1* can be used with any procedure for finding the eigenvalues or eigenvectors of a symmetric tridiagonal matrix (for example [2, 8]).

tred 2 stores also the input matrix A_1 in a square $n \times n$ array and can preserve full information on A_1. Its use is mandatory when all eigenvalues and eigenvectors are to be found via those of A_{n-1} using procedure *tql 2* [2], which is a variant of the QR algorithm. The combination *tred 2* and *tql 2* is probably the most efficient of known methods when the complete eigensystem of A_1 is required. It produces eigenvectors always orthogonal to working accuracy and provides the possibility of "overwriting" the eigenvectors on the columns of A_1. *tred 2* should never be used if eigenvectors are not required.

tred 3 differs from *tred 1* with respect to the storage arrangement. The lower triangle of A_1 has to be stored as a linear array of $\frac{1}{2}n(n+1)$ elements. Its main advantage is in the economy of storage, but it is marginally slower than *tred 1* and the original information is lost. *tred 3* may be used as *tred 1.*

A procedure *tred 4* acting as *tred 2* but with the storage arrangement of *tred 3* may be obtained if the first statement in the body of procedure *tred 2* is replaced by

```
k := 0;
for i := 1 step 1 until n do
for j := 1 step 1 until i do
begin k := k + 1; x[i, j] := a[k] end ij;
```

When *tred 2* is used in conjunction with *tql 2* [2], the eigenvectors of the original matrix, A_1, are automatically provided. *tred 1* and *tred 3* on the other hand are used with procedures which compute the eigenvectors of A_{n-1} and these must be transformed into the corresponding eigenvectors of A_1. The relevant procedures for doing this are *trbak 1* and *trbak 3* respectively; there is no need for a procedure *trbak 2.*

trbak 1 derives eigenvectors *m1* to *m2* of A_1 from eigenvectors *m1* to *m2* of A_{n-1} making use of the details of the transformation from A_1 to A_{n-1} provided as output from *tred 1. trbak 3* derives eigenvectors *m1* to *m2* of A_1 from eigenvectors *m1* to *m2* of A_{n-1} making use of the details of the transformation from A_1 to A_{n-1} provided as output from *tred 3.*

3. Formal Parameter List

3.1 (a) Input to procedure *tred 1*

n order of the real symmetric matrix $A = A_1$.

tol a machine dependent constant. Should be set equal to *eta/macheps* where *eta* is the smallest positive number representable in the computer and *macheps* is the smallest positive number for which

$$1 + macheps \neq 1 .$$

a an $n \times n$ array giving the elements of the symmetric matrix $A = A_1$. Only the lower-triangle of elements is actually used but the upper-triangle of elements is preserved. This makes it possible to compute residuals subsequently if the complete A_1 array is given.

Output from procedure *tred 1*

a an $n \times n$ array; the strict lower-triangle provides information on the Householder transformation; the full upper triangle is the original upper triangle of the input matrix *a*.

d an $n \times 1$ array giving the diagonal elements of the tridiagonal matrix A_{n-1}.

e an $n \times 1$ array of which $e[2]$ to $e[n]$ give the $n-1$ off-diagonal elements of the tridiagonal matrix A_{n-1}. The element $e[1]$ is set to zero.

e2 an $n \times 1$ array such that $e2[i] = (e[i])^2$ for use in procedure *bisect* [1], a call with $e = e2$ is admissible producing no squares.

3.1 (b) Input to procedure *trbak 1*

n order of the real symmetric matrix A_1.

m1, m2 eigenvectors *m1* to *m2* of the tridiagonal matrix A_{n-1} have been found and normalized according to the Euclidean norm.

a the strict lower triangle of this $n \times n$ array contains details of the Householder reduction of A_1 as produced by *tred 1*.

e an $n \times 1$ array giving the $n-1$ off-diagonal elements of the tridiagonal matrix A_{n-1} with $e[1] = 0$, as produced by *tred 1*.

z an $n \times (m2 - m1 + 1)$ array giving eigenvectors *m1* to *m2* of A_{n-1} normalized according to the Euclidean norm.

Output from procedure *trbak 1*

z an $n \times (m2 - m1 + 1)$ array giving eigenvectors *m1* to *m2* of A_1 normalized according to the Euclidean norm.

3.2. Input to procedure *tred 2*

n order of the real symmetric matrix $A = A_1$.

tol machine dependent constant. The same as in *tred 1*.

a an $n \times n$ array giving the elements of the symmetric matrix $A = A_1$. Only the lower-triangle is actually used in *tred 2*. The initial array *a* is copied into the array *z*. If the actual parameters corresponding to *z* and *a* are chosen to be different then the original matrix A_1 is completely preserved, if they are chosen to be identical then A_1 is completely lost.

Output from procedure *tred 2*

a an $n \times n$ array which is identical with the input array *a* unless the actual parameter *z* is chosen to be *a*. In this case the output *a* is the output array *z* described below.

d an $n \times 1$ array giving the diagonal elements of the tridiagonal matrix A_{n-1}.

e an $n \times 1$ array of which $e[2]$ to $e[n]$ give the $n-1$ off-diagonal elements of the tridiagonal matrix A_{n-1}. The element $e[1]$ is set to zero.

z an $n \times n$ array whose elements are those of the orthogonal matrix Q such that $Q^T A_1 Q = A_{n-1}$ (i.e. the product of the Householder transformation matrices.

3.3 (a) Input to procedure *tred 3*

n order of the real symmetric matrix $A = A_1$.

tol machine dependent constant. The same as in *tred 1.*

a a linear array of dimension $\frac{1}{2}n(n+1)$ giving the elements of A_1 in the order a_{11}; a_{21}, a_{22}; ...; $a_{n1}, a_{n2}, \ldots, a_{nn}$.

Output from procedure *tred 3*

a a linear array of dimension $\frac{1}{2}n(n+1)$ giving details of the Householder reduction of A_1.

d an $n \times 1$ array giving the diagonal elements of the tridiagonal matrix A_{n-1}.

e an $n \times 1$ array of which $e[2]$ to $e[n]$ give the $n-1$ off-diagonal elements of the tridiagonal matrix A_{n-1}. The element $e[1]$ is set to zero.

e2 an $n \times 1$ array such that $e2[i] = (e[i])^2$ for use in procedure *bisect* [1], a call with $e = e2$ is admissible producing no squares.

3.3 (b) Input to procedure *trbak 3*

n order of the real symmetric matrix.

m1, m2 eigenvectors *m1* to *m2* of the tridiagonal matrix A_{n-1} have been found and normalized according to the Euclidean norm.

a a linear array of order $\frac{1}{2}n(n+1)$ giving details of the Householder reduction of A_1 as produced by *tred 3.*

z an $n \times (m2 - m1 + 1)$ array giving eigenvectors *m1* to *m2* of A_{n-1} normalized according to the Euclidean norm.

Output from procedure *trbak 3*

z an $n \times (m2 - m1 + 1)$ array giving eigenvectors *m1* to *m2* of A_1 normalized according to the Euclidean norm.

4. ALGOL Programs

```
procedure tred1(n) data:(tol) data and result:(a) result:(d, e, e2);
value n, tol; integer n; real tol; array a, d, e, e2;
comment This procedure reduces the given lower triangle of a symmetric matrix,
        A, stored in the array a[1:n, 1:n], to tridiagonal form using House-
        holder's reduction. The diagonal of the result is stored in the array
        d[1:n] and the sub-diagonal in the last n - 1 stores of the array e[1:n]
```

```
        (with the additional element e[1]=0). e2[i] is set to equal e[i]↑2.
        The lower triangle of the array a, together with the array e, is used
        to store sufficient information for the details of the transformation to
        be recoverable in the procedure trbak1. The strictly upper triangle of
        the array a is left unaltered;
begin   integer i, j, k, l;
        real f, g, h;
        for i := 1 step 1 until n do
        d[i] := a[i, i];
        for i := n step -1 until 1 do
        begin l := i-1; h := 0;
              for k := 1 step 1 until l do
              h := h+a[i, k]×a[i, k];
              comment if h is too small for orthogonality to be guaranteed,
                      the transformation is skipped;
              if h≦tol then
              begin e[i] := e2[i] := 0; go to skip
              end;
              e2[i] := h; f := a[i, i-1];
              e[i] := g := if f≧0 then -sqrt(h) else sqrt(h);
              h := h-f×g; a[i, i-1] := f-g; f := 0;
              for j := 1 step 1 until l do
              begin g := 0;
                    comment form element of A×u;
                    for k := 1 step 1 until j do
                    g := g+a[j, k]×a[i, k];
                    for k := j+1 step 1 until l do
                    g := g+a[k, j]×a[i, k];
                    comment form element of p;
                    g := e[j] := g/h; f := f+g×a[i, j]
              end j;
              comment form K;
              h := f/(h+h);
              comment form reduced A;
              for j := 1 step 1 until l do
              begin f := a[i, j]; g := e[j] := e[j]-h×f;
                    for k := 1 step 1 until j do
                    a[j, k] := a[j, k]-f×e[k]-g×a[i, k]
              end j;
        skip: h := d[i]; d[i] := a[i, i]; a[i, i] := h
        end i
end tred1;

procedure trbak1 (n) data: (m1, m2, a, e) data and result: (z);
value n, m1, m2; integer n, m1, m2; array a, e, z;
comment This procedure performs, on the matrix of eigenvectors, Z, stored in
        the array z[1:n, m1:m2], a backtransformation to form the eigen-
```

vectors of the original symmetric matrix from the eigenvectors of the tridiagonal matrix. The new vectors are overwritten on the old ones.

The sub-diagonal of the tridiagonal matrix must be stored in the array $e[1:n]$, and the details of the Householder reduction must be stored in the lower triangle of the array $a[1:n, 1:n]$, as left by the procedure *tred1*.

If z denotes any column of the resultant matrix Z, then z satisfies $z^T \times z = z\,(\text{input})^T \times z\,(\text{input})$;

```
begin   integer i, j, k, l;
        real h, s;
        for i := 2 step 1 until n do
        if e[i] ≠ 0 then
        begin l := i − 1;  h := e[i] × a[i, i − 1];
              for j := m1 step 1 until m2 do
              begin s := 0;
                    for k := 1 step 1 until l do
                    s := s + a[i, k] × z[k, j];
                    s := s/h;
                    for k := 1 step 1 until l do
                    z[k, j] := z[k, j] + s × a[i, k]
              end j
        end i
end trbak1;
```

```
procedure tred2 (n) data: (tol) data and result: (a) result: (d, e, z);
value n, tol; integer n; real tol; array a, d, e, z;
comment This procedure reduces the given lower triangle of a symmetric matrix,
        A, stored in the array a[1:n, 1:n], to tridiagonal form using House-
        holder's reduction. The diagonal of the result is stored in the array
        d[1:n] and the sub-diagonal in the last n − 1 stores of the array e[1:n]
        (with the additional element e[1] = 0). The transformation matrices
        are accumulated in the array z[1:n, 1:n]. The array a is left unaltered
        unless the actual parameters corresponding to a and z are identical;
begin   integer i, j, k, l;
        real f, g, h, hh;
        for i := 1 step 1 until n do
        for j := 1 step 1 until i do
        z[i, j] := a[i, j];
        for i := n step −1 until 2 do
        begin l := i − 2;  f := z[i, i − 1];  g := 0;
              for k := 1 step 1 until l do
              g := g + z[i, k] × z[i, k];
              h := g + f × f;
              comment if g is too small for orthogonality to be guaranteed,
                      the transformation is skipped;
              if g ≦ tol then
```

```
                begin e[i] := f;  h := 0;  go to skip
                end;
                l := l + 1;
                g := e[i] := if f ≧ 0 then -sqrt(h) else sqrt(h);
                h := h - f × g;  z[i, i - 1] := f - g;  f := 0;
                for j := 1 step 1 until l do
                begin z[j, i] := z[i, j]/h;  g := 0;
                        comment form element of A × u;
                        for k := 1 step 1 until j do
                        g := g + z[j, k] × z[i, k];
                        for k := j + 1 step 1 until l do
                        g := g + z[k, j] × z[i, k];
                        comment form element of p;
                        e[j] := g/h;  f := f + g × z[j, i]
                end j;
                comment form K;
                hh := f/(h + h);
                comment form reduced A;
                for j := 1 step 1 until l do
                begin f := z[i, j];  g := e[j] := e[j] - hh × f;
                        for k := 1 step 1 until j do
                        z[j, k] := z[j, k] - f × e[k] - g × z[i, k]
                end j;
        skip:   d[i] := h
        end i;

        d[1] := e[1] := 0;
        comment accumulation of transformation matrices;
        for i := 1 step 1 until n do
        begin l := i - 1;
                if d[i] ≠ 0 then
                for j := 1 step 1 until l do
                begin g := 0;
                        for k := 1 step 1 until l do
                        g := g + z[i, k] × z[k, j];
                        for k := 1 step 1 until l do
                        z[k, j] := z[k, j] - g × z[k, i]
                end j;
                d[i] := z[i, i];  z[i, i] := 1;
                for j := 1 step 1 until l do
                z[i, j] := z[j, i] := 0
        end i
end tred2;

procedure tred3 (n) data: (tol) data and result: (a) result: (d, e, e2);
value n, tol; integer n; real tol; array a, d, e, e2;
comment This procedure reduces the given lower triangle of a symmetric matrix,
        A, stored row by row in the array a[1 : n(n + 1)/2], to tridiagonal form
```

using Householder's reduction. The diagonal of the result is stored in the array $d[1:n]$ and the sub-diagonal in the last $n-1$ stores of the array $e[1:n]$ (with the additional element $e[1]=0$). $e2[i]$ is set to equal $e[i]\uparrow 2$. The array a is used to store sufficient information for the details of the transformation to be recoverable in the procedure *trbak3*;

```
begin   integer i, iz, j, jk, k, l;
        real f, g, h, hh;
        for i := n step -1 until 1 do
        begin l := i-1; iz := i×l/2; h := 0;
              for k := 1 step 1 until l do
              begin f := d[k] := a[iz+k]; h := h+f×f
              end k;
              comment if h is too small for orthogonality to be guaranteed,
                      the transformation is skipped;
              if h ≦ tol then
              begin e[i] := e2[i] := h := 0; go to skip
              end;
              e2[i] := h;
              e[i] := g := if f ≧ 0 then -sqrt(h) else sqrt(h);
              h := h-f×g; d[l] := a[iz+l] := f-g; f := 0;
              for j := 1 step 1 until l do
              begin g := 0; jk := j×(j-1)/2+1;
                    comment form element of A×u;
                    for k := 1 step 1 until l do
                    begin g := g+a[jk]×d[k];
                          jk := jk+(if k<j then 1 else k)
                    end k;
                    comment form element of p;
                    g := e[j] := g/h; f := f+g×d[j]
              end j;
              comment form K;
              hh := f/(h+h); jk := 0;
              comment form reduced A;
              for j := 1 step 1 until l do
              begin f := d[j]; g := e[j] := e[j]-hh×f;
                    for k := 1 step 1 until j do
                    begin jk := jk+1; a[jk] := a[jk]-f×e[k]-g×d[k]
                    end k
              end j;
        skip: d[i] := a[iz+i]; a[iz+i] := h
        end i
end tred3;
procedure trbak3 (n) data: (m1, m2, a) data and result: (z);
value n, m1, m2; integer n, m1, m2; array a, z;
comment This procedure performs, on the matrix of eigenvectors, Z, stored in
        the array z[:n, m1:m1], a backtransformation to form the eigen-
```

vectors of the original symmetric matrix from the eigenvectors of the tridiagonal matrix. The new vectors are overwritten on the old ones.

The details of the Householder reduction must be stored in the array $a[1:n(n+1)/2]$, as left by the procedure *tred3*.

If z denotes any column of the resultant matrix Z, then z satisfies $z^T \times z = z\,(\text{input})^T \times z\,(\text{input})$;

```
begin   integer i, iz, j, k, l;
        real h, s;
        for i := 2 step 1 until n do
        begin l := i - 1;  iz := i × l/2;  h := a[iz + i];
              if h ≠ 0 then
              for j := m1 step 1 until m2 do
              begin s := 0;
                    for k := 1 step 1 until l do
                    s := s + a[iz + k] × z[k, j];
                    s := s/h;
                    for k := 1 step 1 until l do
                    z[k, j] := z[k, j] - s × a[iz + k]
              end j
        end i
end trbak3;
```

5. Organizational and Notational Details

In general the notation used in the present procedures is the same as that used in the earlier procedure [4]. The main differences in the new versions are outlined in the following discussion.

In the original treatment, after forming σ_i defined by

$$\sigma_i = a_{l1}^2 + \cdots + a_{l,l-1}^2 \quad (l = n - i + 1) \tag{10}$$

the current transformation was skipped if $\sigma_i = 0$. This saved time and also avoided indeterminancy in the elements of the transformation matrix. A more careful treatment here is important.

If A_1 has any multiple roots, then with exact computation at least one σ_i must be zero, since $\sigma_i^1 = e_l$, an off-diagonal element of the tridiagonal matrix. This implies that for some l we must have $a_{lj} = 0$ $(j = 1, \ldots, l-1)$. Hence in practice it is quite common for a complete set of a_{lj} to be very small. Suppose for example on a computer having numbers in the range 2^{-128} to 2^{128} we have at the stage $l = 4$

$$a_{41} = 2^{-63}, \quad a_{42} = 2^{-65}, \quad a_{43} = 2^{-63}. \tag{11}$$

Then $a_{42}^2 = 2^{-130}$ and will probably be replaced by zero. As a result the transformation which is dependent on the ratios of the a_{4j} and σ_{n-3} will differ substantially from an orthogonal matrix.

In the published procedures we have replaced "negligible" σ_i by zero and skipped the corresponding transformation. A computed σ_i is regarded as negligible if

$$\sigma_i \leqq eta/macheps = tol\,(\text{say}) \tag{12}$$

where *eta* is the smallest positive number representable in the computer and *macheps* is the relative machine precision. This ensures that the transformation is skipped whenever there is a danger that it might not be accurately orthogonal.

The maximum error in any eigenvalue as a result of the use of this criterion is $2\times tol^{\frac{1}{2}}$. On a computer with a range 2^{-256} to 2^{256} and a 40 digit mantissa we have

$$2\times tol^{\frac{1}{2}}=2^{-107}.$$

The resulting error would therefore be of little consequence in general; indeed larger errors than this would usually be inevitable in any case as a result of rounding errors.

However, there are some matrices which have eigenvalues which are determined with a very small error compared with $\|A\|_2$. Consider for example the matrix

$$A=\begin{bmatrix} "10^{-12}" & "10^{-12}" & "10^{-12}" \\ "10^{-12}" & "10^{-6}" & "10^{-6}" \\ "10^{-12}" & "10^{-6}" & "1" \end{bmatrix} \tag{13}$$

where "10^{-k}" denotes a number of the order of magnitude 10^{-k}. Such a matrix has an eigenvalue λ of order 10^{-12}. If we assume that the elements of A are prescribed to within one part in 10^t then in general, the eigenvalue λ will itself be determined to within a few parts in 10^t, that is with an absolute error of order 10^{-12-t}.

Clearly in order to deal with such matrices satisfactorily one would wish to make the criterion for skipping transformations as strict as possible. The range of permissable σ_i could be extended without losing orthogonality by proceeding as follows. Let

$$m_i=\max_{j=1}^{l-1}|a_{lj}|, \qquad r_{lj}=a_{lj}/m_i \quad (j=1,\ldots,l-1). \tag{14}$$

Then the transformation may be determined using the r_{lj} instead of the a_{lj} and we skip the transformation only if $m_i=0$. We have decided in favour of the simpler technique described earlier. It should be appreciated that if the eigenvalues of the tridiagonal matrix are to be found by the procedure *bisect* [1] it is the squares of the subdiagonal elements which are used, i.e. the σ_i themselves. Hence when $\sigma_i<eta$ the corresponding e_l^2 is necessarily replaced by zero and we cannot reduce the bound for the errors introduced in the eigenvalues below $2(eta)^{\frac{1}{2}}$. For the computer specified above even the extreme precautions merely reduce the error bound from 2^{-107} to 2^{-127}.

When dealing with matrices having elements of widely varying orders of magnitude it is important that the smaller elements be in the top left-hand corner. The matrix should *not* be presented in the form

$$\begin{bmatrix} "1" & "10^{-6}" & "10^{-12}" \\ "10^{-6}" & "10^{-6}" & "10^{-12}" \\ "10^{-12}" & "10^{-12}" & "10^{-12}" \end{bmatrix}. \tag{15}$$

This is because the Householder reduction is performed starting from the last row and the consequences this has for rounding errors. To give maximum accuracy

when dealing with matrices of type (13) all inner products are formed in direction of ascending subscripts.

The procedure *tred 2* is designed specifically for use with the procedure *tql 2* [2] when all eigenvalues and eigenvectors are required. With this combination it is more economical to have the orthogonal transformation Q reducing A_1 to tridiagonal form before solving the eigenproblem of A_{n-1}. We have

$$Q^T A_1 Q = A_{n-1} \tag{16}$$

and

$$Q = P_1 P_2 \dots P_{n-2}. \tag{17}$$

The product Q is computed after completing the Householder reduction and it is derived in the steps

$$\begin{aligned} Q_{n-2} &= P_{n-2} \\ Q_i &= P_i Q_{i+1} \quad i = n-3, \dots, 1 \\ Q &= Q_1. \end{aligned} \tag{18}$$

Because of the nature of the P_i the matrix Q_i is equal to the identity matrix in rows and columns $n, n-1, \dots, n-i+1$. Hence it is possible to overwrite the relevant parts of successive Q_i on the same $n \times n$ array which initially stores details of the transformation $P_1, \dots, P_{n-2}$. The details of u_i are stored in row l of the $n \times n$ array; it proves convenient to store also u_i/H_i in column l of the $n \times n$ array. If information on the original matrix A_1 is required for subsequent calculation of residuals, A_1 must be copied before starting the Householder reduction and multiplying the transformation matrices. The working copy is the array z. If A_1 is not required for subsequent use, the $n \times n$ extra storage locations may be saved by taking $z = a$, in which case A_1 is effectively "overwritten on itself".

In all three procedures *tred 1, tred 2, tred 3* the non-zero elements of each p_i are stored in the storage locations required for the vector e. At each stage the number of storage locations not yet occupied by elements of e is adequate to store the non-zero elements of p_i. The vector p_i is subsequently overwritten with q_i.

tred 2 contains one feature which is not included in the other two procedures. If, at any stage in the reduction, the row which is about to be made tridiagonal is already of this form, the current transformation can be skipped. This is done in *tred 2* since the economy achieved is more worthwhile. If *tred 2* is provided with a matrix which is already entirely tridiagonal then virtually no computation is performed.

The procedure *trbak 1* and *trbak 3* call for little comment. The details of the transformations from A_1 to A_{n-1} are provided in the outputs a and e from *tred 1* and *tred 3* respectively.

6. Discussion of Numerical Properties

The procedures *tred 1, tred 2, tred 3* are essentially the same apart from the organization and give a very stable reduction. It has been shown [3, 6] that the computed tridiagonal matrix is orthogonally similar to $A + E$ where

$$\|E\|_F \leq k_1 n^2 2^{-t} \|A\|_F \tag{19}$$

where k_1 is a constant depending on the rounding procedure, t is the number of binary digits in mantissa and $\|\cdot\|_F$ denotes the Frobenius norm. If an inner product procedure which accumulates without rounding is used when computing σ_i, p_i and K_i then

$$\|E\|_F \leqq k_2\, n\, 2^{-t} \|A\|_F \tag{20}$$

where k_2 is a second constant. Both these bounds, satisfactory though they are, prove to be substantial overestimates in practice. The bounds ignore the problem of underflow, but the additional error arising from this cause when the criterion (12) is used is usually negligible compared with these bounds.

The error involved in using *trbak 1* and *trbak 3* to recover an eigenvector of A_1 from a computed eigenvector of computed A_{n-1} may be described as follows. Suppose that by any method we have obtained a computed normalized eigenvector z of A_{n-1} and can show that it is an exact eigenvector of $A_{n-1}+F$ with a known bound for $\|F\|_F$. If $\bar{x}$ is the corresponding computed eigenvector of A_1 then

$$\bar{x} = x + f$$

where x is an exact eigenvector of A_1+G with some G satisfying

$$\|G\|_F \leqq \|E\|_F + \|F\|_F,$$

(E being as defined above) and f satisfies

$$\|f\|_2 \leqq k_3\, n\, 2^{-t}.$$

In practice the bound for $\|F\|_F$ is usually smaller than that for $\|E\|_F$ and easily the most important source of error in x is that due to the inherent sensitivity of the relevant eigenvector to the perturbation E in A_1.

Again the computed transformation $\bar{Q}$ given by *tred 2* satisfies the relations

$$\bar{Q} = Q + R, \quad \|R\|_F \leqq k_4 n^2 2^{-t}$$

where Q is the exact orthogonal matrix such that

$$Q^T (A_1 + E)\, Q = A_{n-1}$$

and again the most important consideration is the effect of the perturbation E.

7. Test results

As a formal test of the procedures described here the matrices

$$A = \begin{bmatrix} 10 & 1 & 2 & 3 & 4 \\ 1 & 9 & -1 & 2 & -3 \\ 2 & -1 & 7 & 3 & -5 \\ 3 & 2 & 3 & 12 & 1 \\ 4 & -3 & -5 & -1 & 15 \end{bmatrix} \quad \text{and} \quad B = \begin{bmatrix} 5 & 1 & -2 & 0 & -2 & 5 \\ 1 & 6 & -3 & 2 & 0 & 6 \\ -2 & -3 & 8 & -5 & -6 & 0 \\ 0 & 2 & -5 & 5 & 1 & -2 \\ -2 & 0 & -6 & 1 & 6 & -3 \\ 5 & 6 & 0 & -2 & -3 & 8 \end{bmatrix}$$

were used. The second matrix was chosen so as to illustrate the case of multiple eigenvalues. The use of the procedures on more sophisticated examples will be illustrated in later papers describing the solution of the eigenproblem for tri-

diagonal matrices [2, 7]. The results obtained on KDF 9, a computer using a 39 binary digit mantissa, are given in Tables 1–3.

In Table 1 we give the subdiagonal and diagonal of the tridiagonal matrices determined using *tred 1*, *tred 2* and *tred 3* and the squares of the subdiagonal elements determined using *tred 1* and *tred 3* (*tred 2* does not output this vector). Procedures *tred 1* and *tred 3* always give identical results. The results obtained with *tred 2* may differ for three reasons. First the signs of sub-diagonal elements may differ because *tred 2* skips any step in which all elements to be annihilated are zero. Secondly if a σ_i is negligible it is replaced by zero in *tred 1* and *tred 3* but not in *tred 2*. Thirdly the arithmetic operations are performed in a slightly different order in *tred 2* and because the digital operations are, in general, non-commutative the rounding errors may be different.

As far as matrix A is concerned results obtained on a computer with a different word length and different rounding procedures should agree with those given almost to full working accuracy. For matrix B, however, different tridiagonal matrices may be obtained on different computers since the determination of the elements of the tridiagonal matrix itself is not stable when there are multiple roots. Nevertheless the *eigenvalues* should be preserved almost to full working accuracy. Note that with matrix B the element e_4 is of order 10^{-11}, that is of the order $macheps \times \|B\|_2$. With exact computation it should be zero. On KDF 9, e_4 was not small enough for the transformation to be skipped.

Table 1

Matrix A, tred 1 and tred 3

Sub-diagonal	Diagonal	Sub-diagonal squared
$+0.00000000000;$	$+9.29520217754_{10}+0$	$+0.00000000000$
$+7.49484677741_{10}-1$	$+1.16267115560_{10}+1$	$+5.61727282169_{10}-1$
$-4.49626820120_{10}+0$	$+1.09604392078_{10}+1$	$+2.02164277371_{10}+1$
$-2.15704099085_{10}+0$	$+6.11764705885_{10}+0$	$+4.65282583621_{10}+0$
$+7.14142842854_{10}+0$	$+1.50000000000_{10}+1$	$+5.10000000000_{10}+1$

Matrix A, tred 2

Sub-diagonal	Diagonal
$+0.00000000000$	$+9.29520217754_{10}+0$
$-7.49484677741_{10}-1$	$+1.16267115560_{10}+1$
$-4.49626820120_{10}+0$	$+1.09604392078_{10}+1$
$-2.15704099085_{10}+0$	$+6.11764705885_{10}+0$
$+7.14142842854_{10}+0$	$+1.50000000000_{10}+1$

Matrix B, tred 1 and tred 3

Sub-diagonal	Diagonal	Sub-diagonal squared
$+0.00000000000$	$+4.40021275393_{10}+0$	$+0.0000000000$
$-7.83768800584_{10}-1$	$+3.80347681618_{10}+0$	$+6.14293532768_{10}-1$
$-8.10502269777_{10}+0$	$+1.07963104302_{10}+1$	$+6.56913929312_{10}+1$
$-2.97985867006_{10}-11$	$+4.25675675677_{10}+0$	$+8.87955769356_{10}-22$
$-1.92466782539_{10}+0$	$+6.74324324323_{10}+0$	$+3.70434623811_{10}+0$
$+8.60232526704_{10}+0$	$+8.00000000000_{10}+0$	$+7.40000000000_{10}+1$

Matrix B, tred 2

Sub-diagonal	Diagonal
$+0.00000000000$	$+4.40021275387_{10}+0$
$+7.83768800584_{10}-1$	$+3.80347681618_{10}+0$
$-8.10502269777_{10}+0$	$+1.07963104303_{10}+1$
$-2.97985867006_{10}-11$	$+4.25675675677_{10}+0$
$-1.92466782539_{10}+0$	$+6.74324324323_{10}+0$
$+8.60232526704_{10}+0$	$+8.00000000000_{10}+0$

In Table 2 we give the eigenvalues of matrices A and B. The values presented are those obtained via *tred 1* and procedure *bisect* [1]. The values obtained with *tred 3* were, of course, identical with these, while those obtained using *tred 2* and *tql2* differed by a maximum of 2 *macheps* $\|\cdot\|_2$.

Table 2. *Eigenvalues*

Matrix A	Matrix B
$+1.65526620775_{10}+0$	$-1.59873429358_{10}+0$
$+6.99483783064_{10}+0$	$-1.59873429346_{10}+0$
$+9.36355492016_{10}+0$	$+4.45598963847_{10}+0$
$+1.58089207645_{10}+1$	$+4.45598963855_{10}+0$
$+1.91754202773_{10}+1$	$+1.61427446551_{10}+1$
	$+1.61427446553_{10}+1$

Finally in Table 3 we give the transformation matrices derived using *tred 2*. It will be observed that the last row and column are e_n^T and e_n respectively.

Table 3. *Transformation matrix for matrix A*

$-3.84541619403_{10}-2$	$-8.26959819664_{10}-1$	$+3.05490408455_{10}-2$
$-8.68565336976_{10}-1$	$-2.40190402438_{10}-1$	$+1.06921643009_{10}-1$
$+4.49244356071_{10}-1$	$-5.03710280447_{10}-1$	$-2.32936436565_{10}-1$
$+2.05657582815_{10}-1$	$-6.87166691116_{10}-2$	$+9.66113417195_{10}-1$
$+0.00000000000$	$+0.00000000000$	$+0.00000000000$
$+5.60112033610_{10}-1$	$+0.00000000000$	
$-4.20084025208_{10}-1$	$+0.00000000000$	
$-7.00140042012_{10}-1$	$+0.00000000000$	
$-1.40028008398_{10}-1$	$+0.00000000000$	
$+0.00000000000$	$+1.00000000000_{10}+0$	

Transformation matrix for matrix B

$+7.34899893569_{10}-1$	$-2.80793152869_{10}-1$	$-6.24667838579_{10}-2$
$-5.99285559556_{10}-1$	$-3.02902839915_{10}-1$	$+1.37699891160_{10}-1$
$-1.43891033659_{10}-1$	$-1.59317058391_{10}-1$	$-9.76684926340_{10}-1$
$-2.22624697547_{10}-1$	$-5.58793489748_{10}-1$	$+1.23948910869_{10}-1$
$+1.74678501867_{10}-1$	$-7.01265274773_{10}-1$	$+8.86558686443_{10}-2$
$+0.00000000000$	$+0.00000000000$	$+0.00000000000$
$-1.98336619947_{10}-1$	$+5.81238193719_{10}-1$	$+0.00000000000$
$+2.08947221014_{10}-1$	$+6.97485832461_{10}-1$	$+0.00000000000$
$+0.00000000000$	$+0.00000000000$	$+0.00000000000$
$-7.54168875847_{10}-1$	$-2.32495277487_{10}-1$	$+0.00000000000$
$+5.90112659345_{10}-1$	$-3.48742916231_{10}-1$	$+0.00000000000$
$+0.00000000000$	$+0.00000000000$	$+1.00000000000_{10}+0$

The tests were confirmed at the Technische Hochschule München.

Acknowledgements. The work of R. S. MARTIN and J. H. WILKINSON has been carried out at the National Physical Laboratory and that of C. REINSCH at the Technische Hochschule München.

References

1. BARTH, W., R. S. MARTIN, and J. H. WILKINSON: Calculation of the eigenvalues of a symmetric tridiagonal matrix by the method of bisection. Numerische Mathematik **9**, 386—393 (1967).
2. BOWDLER, H., C. REINSCH, R. S. MARTIN, and J. H. WILKINSON: The QL algorithm for symmetric tridiagonal matrices. (To appear in this series.)
3. ORTEGA, J. M.: An error analysis of HOUSEHOLDER's method for the symmetric eigenvalue problem. Numerische Mathematik **5**, 211—225 (1963).
4. WILKINSON, J. H.: HOUSEHOLDER's method for symmetric matrices. Numerische Mathematik **4**, 354—361 (1962).
5. — Calculation of the eigenvectors of a symmetric tridiagonal matrix by inverse iteration. Numerische Mathematik **4**, 368—376 (1962).
6. — The algebraic eigenvalue problem. London: Oxford University Press 1965.
7. — Calculation of the eigenvectors of a symmetric tridiagonal matrix by inverse iteration. (Improved version to appear in this series.)
8. REINSCH, C., and F. L. BAUER: Rational QR transformation with Newton shift for symmetric tridiagonal matrices. Numerische Mathematik **11**, 264—272 (1968).

ROGER S. MARTIN
Dr. J. H. WILKINSON
National Physical Laboratory
Mathematics Division
Teddington, Middlesex
Great Britain

Dr. C. REINSCH
Mathematisches Institut
der Technischen Hochschule
8000 München 2, Arcisstr. 21